

ECE 220 Computer Systems & Programming

Lecture 5 — Introduction to C

September 8, 2026

Prof. Sayan Mitra mitras@illinois.edu Office hours: TBD
Course website: courses.grainger.illinois.edu/ece220/fa2026

The Productivity Problem

**You can make the LC-3 do anything —
if you have all afternoon.**

```
AND R0, R0, #0  
AND R1, R1, #0  
ADD R1, R1, #5  
... 11 more lines ...  
BRp LOOP
```

multiply, in LC-3 (Lecture 1)



```
y = x * 5;
```

the same program, in C

Today's Two Questions

**One line of C becomes a dozen lines of assembly —
someone has to do that translation.**

1. What must a language provide so we can stop thinking in registers?
2. What happens between the C you write and the bits that run?

You have earned this lecture: after four weeks of LC-3 you know exactly what C is saving you from.

Why a Higher-Level Language?

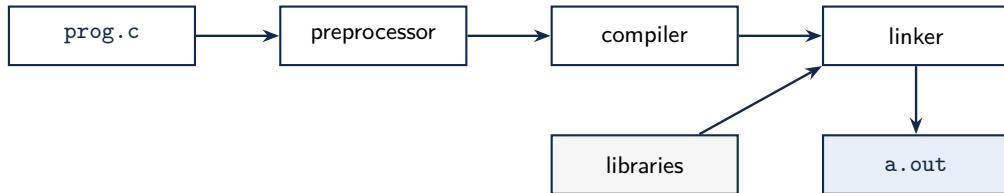
- **Symbolic names** for values — no more deciding which register or memory location
- **Abstraction of the hardware** — you can write `a = b * c;` even though the LC-3 has no multiply instruction
- **Expressiveness** — meaningful identifiers; one-line patterns for if-then-else, loops
- **Readability** — code is read far more often than it is written
- **Safeguards** — rules enforced at compile time and run time

Characteristics of C

- C is a **procedural** language: an explicit sequence of steps, organized into *functions* (a.k.a. subroutines!)
- C is **compiled**, not interpreted: a compiler translates the whole program to machine code ahead of time (interpreted languages like MATLAB and Python are executed by another program)
- C is **statically typed**: every expression's type is checked at compile time

In-class question: after `int x = 3.411;` what is the value of `x`? _____

From Source to Executable



- **Preprocessor:** macro substitution (`#define`), file inclusion (`#include`) — output is still C
- **Compiler:** translates C to machine instructions (an *object file*)
- **Linker:** combines object files and libraries into the executable
- `gcc` runs all three — one command from source to program

Anatomy of a C Program

```
/* My first C program: print the value of pi. */
#include <stdio.h>
#define PI 3.1416f

int main() {
    float pi = PI;
    printf("pi = %f\n", pi);
    return 0;
}
```

- a. Comment (*/* ... */*)
- b. Preprocessor directives (`#include`, `#define`)
- c. The `main` function — execution starts here
- d. Variable declaration: **type**, **identifier**, **scope**
- e. I/O (`printf`)
- f. Return value
- g. Every statement ends with _____

Variables: Types, Scope, Storage

Types

- `int` (long, unsigned; hex literals like `0xD`)
- `float`, `double`
- `char` (single quotes: `'M'`)
- Qualifiers: `const`, `static`

Scope: local vs. global — *where* the name is visible

Storage class: automatic vs. static — *how long* the value lives



Operators

- **Assignment** = `—` and its evil twin `==` (equality). Mixing them up compiles fine and ruins your day.
- **Order of evaluation:**
 - precedence: `x = 2+3*4;`
 - associativity: `x = 2+3-4+5;`
 - parentheses: `x = a*(b + c)*d/2;`
- **Logical:** _____
- **Bitwise:** _____
- **Relational:** _____

`x = _____``x = _____`

Full precedence table: Patt & Patel Table 12.5.

Operators, Continued

Increment / decrement (pre vs. post):

```
x = 4; y = ++x;
```

```
x = 4; y = x++;
```

x = _____, y = _____

x = _____, y = _____

Conditional (ternary):

```
x = (y < z) ? 5 : 7;
```

if $y < z$ then $x =$ _____; otherwise $x =$ _____

Compound assignment: $a += b; \equiv$ _____

Basic I/O: printf and scanf

```
#include <stdio.h>
```

header for standard I/O

printf — print to screen:

```
printf("%d is prime", 43);  
printf("sum=%d\n", a+b);  
printf("%d+%d=%d\n", a, b, a+b);
```

scanf — read user input:

```
scanf("%c", &nextchar);  
scanf("%f", &radius);  
scanf("%d %d", &len, &ht);
```

Format specifiers

%d	decimal int
%x	hexadecimal
%c	character
%s	string
%f	float
%lf	double
\n	newline

Why the & in scanf? Hold that thought — pointers are coming.

man printf, man scanf — the manual pages are the reference.

In-Class Problem: Bit Gymnastics

Declare `x`, `y`, `z`; set `x` to 3 and `y` to `x` squared; shift `y` left by `x` bits; OR `x` and `y` into `z`; print `z`.

- Which operators? `*` `<<` `|`
- Predict the output before you run it!

```
/* Bit gymnastics: shifts and bitwise OR */
#include <stdio.h>

int main() {
    /* declare integer variables x, y, z */

    /* set x to 3, y to x squared */

    /* left shift y by x bits */

    /* bitwise OR of x and y into z */

    /* print z */

    return 0;
}
```

In-Class Problem: Fahrenheit to Celsius

Read a temperature in Fahrenheit; print it in Celsius:

$$C = (F - 32) * 5 / 9$$

- What type for the variables?
- What does `scanf` need in front of the variable?
- Why not write $(F-32)*(5/9)$?

```
/* Fahrenheit to Celsius: C = (F-32)*5/9 */

/* preprocessor directives */

int main() {
    /* declare variables (as float) */

    /* prompt the user */

    /* read the input (careful!) */

    /* compute Celsius */

    /* print the result */

    return 0;
}
```

Summary & What's Next

Today

- **Why C:** symbolic names, hardware abstraction, expressiveness, safeguards — the compiler writes the assembly you have been writing by hand
- **The pipeline:** preprocessor → compiler → linker; `gcc` runs it all
- **Variables:** type, identifier, scope, storage class — locals live on the run-time stack
- **Operators and I/O:** precedence, `=` vs `==`, `printf/scanf`

Next lecture

- Control flow in C: `if`, loops, `switch` — and compiling on EWS with `gcc`

Code from today: `first.c`, `ex1.c`, `ex2.c` in the course code repository.